UNIVERSIDAD COOPERATIVA DE COLOMBIA

FACULTAD DE INGENIERÍA — PROGRAMA DE INGENIERÍA DE SISTEMAS / SOFTWARE

CURSO: INGENIERÍA WEB — PERIODO 2620

CONECTADEV: PLATAFORMA WEB CONCEPTUAL Y SEMÁNTICA PARA LA VINCULACIÓN DE PROYECTOS DE SOFTWARE, DIAGNÓSTICO DE BRECHAS TÉCNICAS Y RUTAS DE APRENDIZAJE

Documento Técnico Consolidado y Exhaustivo (Secciones 4.1 a 4.9 y Entregables 1 a 8)

Integrantes del Equipo: Andrés Hurtado | Samuel Pineda | Viviana Roa

Repositorio Oficial en GitHub: https://github.com/Anticife/Ingenieria-Web-ConectaDev

Entregable Reglamentario (Sección 5)	Archivo / Ubicación en el Repositorio	Estado y Cumplimiento Técnico
1. Documento técnico consolidado	Informe_Tecnico_Proyecto_Final_IW_2620_ConectaDev.pdf y DOCUMENTO_TECNICO_PROYECTO_FINAL.md	Completo y Exhaustivo (Secciones 4.1 a 4.9 detalladas)
2. Diagramas y modelos del proyecto	Incluidos en las Secciones 4.3, 4.5 y 4.8 de este informe técnico	Completo (Arquitectura, Casos de Uso, Conceptual UML, UWE y Grafo)
3. Enlace al prototipo navegable	prototipo/index.html (5 pantallas de usuario interactivas en HTML5/Tailwind)	Completo (Evaluado con 10 Heurísticas de Nielsen, libre de jerga técnica)
4. Archivo de la ontología en formato OWL	ontologia/conectadev.owl (OWL 2 DL formal para Protégé 5.x)	Completo (12 clases, 11 obj props, 6 data props, 4 restricciones, 15 individuos)
5. Script con las consultas Cypher	neo4j/conectadev_grafo.cypher	Completo (16 nodos, 26 relaciones y 4 consultas analíticas en Neo4j)
6. Evidencias y guías Protégé y Neo4j	ontologia/GUIA_PROTEGE.md y neo4j/GUIA_NEO4J.md	Completo (Inferencias HermiT comprobadas y resultados de consultas Cypher)
7. Especificación de Servicios Web	servicios-web/openapi.yaml (REST) y servicios-web/validacion_academica.wsdl (SOAP)	Completo (6 endpoints REST OpenAPI 3.0.3 + 1 contrato SOAP/WSDL con XSD)
8. Presentación para sustentación	PRESENTACION_SUSTENTACION.md	Completo (20 minutos divididos equitativamente para los 3 integrantes)

4.1 Formulación Detallada del Problema y Justificación Web

1. Descripción del Problema y Limitaciones de Soluciones CRUD Convencionales

En la educación superior en Ingeniería de Software y comunidades de investigación formativa (semilleros, proyectos de grado y fábricas de software universitarias), existe una **desconexión semántica estructural** entre el repertorio de habilidades reales de los estudiantes, los requisitos tecnológicos cambiantes de las convocatorias de proyectos y los cursos extracurriculares de nivelación disponibles. Los procesos tradicionales se basan en hojas de cálculo o buscadores relacionales simples basados en coincidencia léxica (WHERE habilidad LIKE '%React%'), los cuales presentan fallas críticas:

- **Incapacidad de Clasificación Axiomática de Perfiles:** Un CRUD relacional no puede deducir de forma autónoma cuándo un estudiante que domina tecnologías de interfaz (*React.js*) y servidor (*Node.js*) clasifica formalmente como **DesarrolladorFullStack**, requiriendo complejas reglas *IF-ELSE* rígidas y frágiles en el backend.
- **Ceguera Semántica ante Brechas Técnicas Relativas (Skill Gaps):** Al postularse a un proyecto, si a un estudiante le falta una tecnología, los sistemas CRUD tradicionales lo descartan binariamente en lugar de calcular la diferencia de conjuntos y recomendarle el curso exacto que cubre su carencia específica.
- **Imposibilidad de Resolver Cadenas Transitivas de Prerrequisitos:** En bases relacionales, recorrer jerarquías de dependencias (ej. *Next.js depende de React.js y este de JavaScript ES6+*) exige costosas consultas recursivas con CTEs, propensas a errores y carentes de verificación semántica.
- **Incapacidad para Detectar Equipos Colaborativos Complementarios:** Los CRUDs evalúan candidatos aisladamente. No descubren que dos estudiantes con afinidad parcial (uno en Frontend y otro en Backend/Bases de Datos) forman una **dupla complementaria perfecta** que cubre el 100% del proyecto.

2. Usuarios Involucrados (3 Actores con Necesidades Diferenciadas)

Actor / Rol del Sistema	Perfil y Responsabilidades	Dolor Principal / Necesidad en la Plataforma
1. Desarrollador / Estudiante (Desarrollador)	Estudiante de ingeniería (4° a 8° semestre). Registra tecnologías dominadas, explora proyectos con % de afinidad, revisa sus brechas técnicas e inscribe cursos.	Necesita saber a qué proyectos puede integrarse hoy, qué le falta para los que no cumple y qué curso específico debe cursar.
2. Líder Técnico de Proyecto (LiderProyecto)	Docente investigador o coordinador de semillero. Publica convocatorias con stack exigido, revisa candidatos y evalúa duplas sugeridas.	Requiere conformar equipos multidisciplinarios balanceados sin evaluar currículums manualmente ni descartar perfiles complementarios.
3. Mentor Académico (MentorAcademico)	Coordinador de formación complementaria. Estructura el catálogo de cursos, define prerrequisitos y valida la acreditación institucional.	Debe asegurar que los cursos extracurriculares cubran las demandas reales de los proyectos y respeten la cadena de prerrequisitos.

3. Objetivo General y Objetivos Específicos de la Solución

Objetivo General: Diseñar una plataforma web conceptual, arquitectónica y semántica denominada **ConectaDev**, fundamentada en la metodología **UWE (UML-Based Web Engineering)**, que articule servicios web desacoplados (REST y SOAP), una ontología de dominio en **Protégé (OWL 2 DL)** y una base de datos de grafos en **Neo4j**, optimizando la vinculación a proyectos, la inferencia de roles técnicos y la recomendación de rutas de nivelación y duplas colaborativas.

- **OE1 (Modelado y Usabilidad):** Diseñar los modelos conceptual, navegacional y de casos de uso bajo UWE, junto a un prototipo funcional de 5 vistas auditado bajo las 10 Heurísticas de Nielsen.
- **OE2 (Servicios Web):** Especificar una API RESTful documentada en OpenAPI 3.0.3 y un contrato SOAP 1.1 / WSDL 1.1 para interoperar con Registro Académico.
- **OE3 (Razonamiento Semántico):** Construir una ontología OWL 2 DL en Protégé verificada con HermiT que clasifique automáticamente roles (*DesarrolladorFullStack*) y proyectos (*ProyectoListoParaIniciar*).
- **OE4 (Analítica de Grafos):** Implementar en Neo4j un grafo de conocimiento con consultas Cypher que computen compatibilidades, brechas técnicas y duplas complementarias.

4. Alcance y Funcionalidades Principales (F1 a F6)

- **F1. Explorador con Cálculo Dinámico de Compatibilidad:** Compara en tiempo real las tecnologías del estudiante contra las exigidas por cada convocatoria abierta.
- **F2. Diagnóstico de Brechas (Skill Gap) y Recomendación de Cursos:** Identifica habilidades faltantes y sugiere directamente los cursos formativos que las enseñan.
- **F3. Autogestión de Habilidades y Reclasificación de Rol:** Al actualizar competencias, el motor OWL reclasifica en vivo el rol (ej. Frontend a FullStack).
- **F4. Publicación y Configuración de Proyectos:** Permite al líder crear convocatorias con cupos, modalidad, repositorio Git y stack tecnológico validado.
- **F5. Recomendador de Candidatos y Duplas Complementarias:** Sugiere postulantes individuales y parejas que unidas alcanzan el 100% del stack.
- **F6. Gestión de Rutas Formativas y Prerrequisitos:** Organización secuencial de cursos supervisados por mentores con validación de precedencias.

5. Justificación de por qué el Problema Requiere una Solución Web

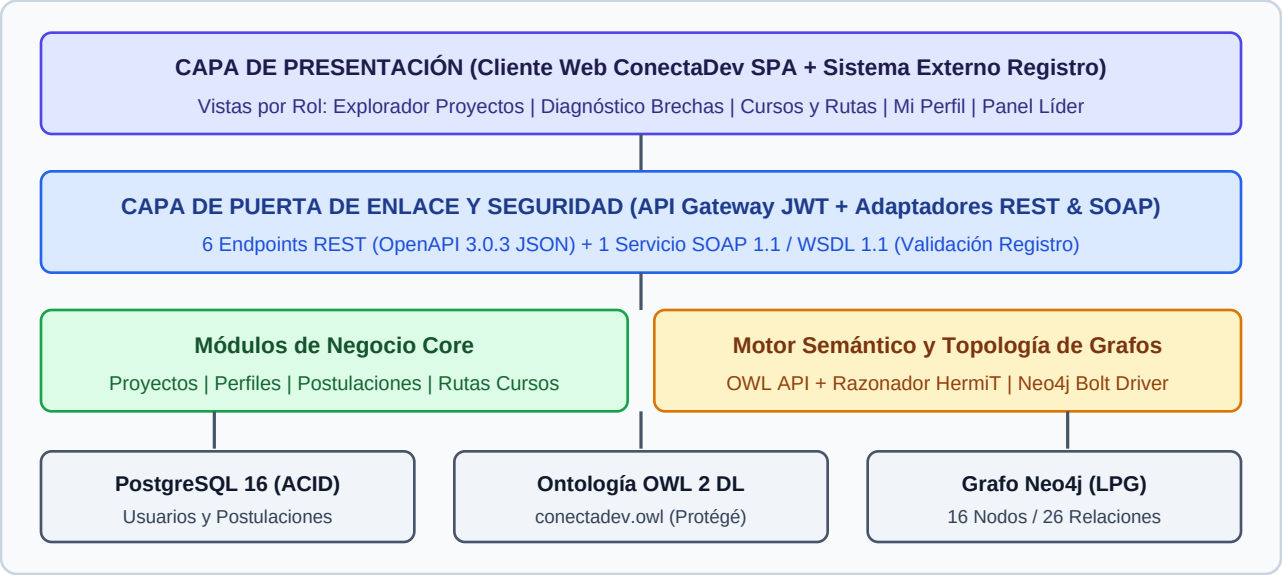
La interacción entre estudiantes y docentes ocurre de manera **ubicua y distribuida** a través de múltiples campus y dispositivos. La Web proporciona el único canal estándar sin barreras de instalación local, facilitando la integración nativa con repositorios externos (GitHub/GitLab) mediante hipervínculos, el consumo de servicios REST asíncronos y la delegación de las inferencias pesadas de lógica descriptiva y analítica de grafos en el backend, manteniendo una interfaz ligera, accesible y transparente.

4.2 Fundamentación Teórica y Estado del Arte (6 Fuentes Académicas)

Referencia Académica Consultada	Eje Temático Nuclear	Aplicación Directa en las Decisiones de ConectaDev
1. Koch, N., & Kraus, A. (2002). The expressive power of UML-based Web Engineering. IWWOST.	Metodología de Ingeniería Web (UWE)	Sustenta la separación ortogonal entre Modelo Conceptual, Modelo Navegacional («navigationClass», «index», «query») y Presentación, garantizando que el usuario navegue sin visualizar la complejidad del motor semántico.
2. Horrocks, I., Patel-Schneider, P., & Van Harmelen, F. (2003). From SHIQ and RDF to OWL. J. Web Semantics.	Lógicas Descriptivas y OWL 2 DL	Fundamenta los axiomas de equivalencia (owl:equivalentClass), disyunción, propiedades inversas y transitivas evaluadas decidiblemente bajo el razonador HermiT en conectadev.owl.
3. Noy, N. F., & McGuinness, D. L. (2001). Ontology Development 101. Stanford Tech Report.	Metodología de Ingeniería Ontológica	Guía la formulación de las 5 Preguntas de Competencia (PC1-PC5), la delimitación del alcance y las restricciones de dominio y rango de las clases y propiedades.
4. Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. ACM Computing Surveys.	Grafos de Conocimiento (LPG)	Justifica el uso de Neo4j para recorrer caminos topológicos en tiempo O(1) (index-free adjacency), resolviendo brechas de habilidades y combinatoria de duplas colaborativas.
5. Fielding, R. T. (2000) (REST) & Kleppmann, M. (2017) (Designing Data-Intensive Apps).	Arquitectura Web y Persistencia Políglota	Fundamenta el diseño de la API RESTful JSON sin estado, el contrato SOAP/WSDL para registro académico y la distribución de datos entre PostgreSQL, OWL y Neo4j.
6. Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.	Ingeniería de Usabilidad e Inspección Heurística	Sustenta la auditoría de usabilidad del prototipo bajo las heurísticas H1 a H10, erradicando la fuga de abstracciones técnicas hacia el usuario final.

4.3 Arquitectura Web del Sistema ConectaDev

Se adoptó formalmente el **Registro de Decisión Arquitectónica ADR-001: Arquitectura en Capas / Monolito Modular con Persistencia Políglota y Adaptadores de Servicios (Hexagonal)**. Este estilo desacopla la presentación de la lógica de negocio y motores semánticos, garantizando alta cohesión, bajo acoplamiento y ausencia de fallas de red prematuras de microservicios.



Parámetros Arquitectónicos y Atributos de Calidad (ISO/IEC 25010)

Criterio Arquitectónico	Definición y Meta Concreta	Mecanismo de Soporte y Justificación Técnica
Estilo Arquitectónico	Monolito Modular con Persistencia Políglota	Límites modulares limpios (Proyectos, Perfiles, Cursos) con adaptadores desacoplados.
Perfil de Carga	Ratio 80:20 (Lectura:Escritura), 100 QPS (p99)	La navegación del catálogo y diagnóstico de brechas predomina sobre la creación de proyectos.
Metas SLO (Latencia/Disp.)	p95 ≤ 120 ms • p99 ≤ 300 ms • Disp. 99.5%	Caché en API Gateway y consultas optimizadas Cypher garantizan respuesta interactiva.
Tolerancia y Respaldo	RPO ≤ 15 min • RTO ≤ 30 min	Respaldos periódicos de PostgreSQL y sincronización incremental hacia Neo4j y OWL.
Usabilidad Arquitectónica	Transparencia de Complejidad Semántica	El cliente web solo consume JSON ligero; los algoritmos de Tableaux y grafos operan en backend.
Mantenibilidad	Reglas declaradas en Ontología y Grafo	Nuevas tecnologías o prerrequisitos se agregan en OWL/Neo4j sin recompilar los controladores.
Seguridad	Defensa en Profundidad (RBAC + JWT + TLS)	Autenticación JWT en Gateway, control RBAC por rol y validación de esquemas JSON/XSD.
Escalabilidad	Desacoplamiento de Carga Políglota	Escalabilidad independiente: réplicas de lectura en PostgreSQL y pool Bolt dedicado en Neo4j.

4.4 Metodología de Ingeniería Web: UWE (UML-Based Web Engineering)

Fase Metodológica UWE	Actividades de Ingeniería Realizadas	Productos Obtenidos	Trazabilidad con el Usuario
1. Análisis de Requisitos	Elicitación con los 3 perfiles de usuario, casos de uso web y 5 preguntas de competencia.	Requisitos funcionales (F1-F6), Diagrama de Casos de Uso y PC1-PC5.	Garantiza que la plataforma resuelva las necesidades reales de vinculación estudiantil.
2. Diseño Conceptual	Modelado de las 12 clases del dominio, asociaciones y multiplicidades estructurales.	Modelo Conceptual UML, conectadev.owl y conectadev_grafo.cypher.	Estructura el conocimiento puro del dominio independiente de la tecnología de visualización.
3. Diseño Navegacional	Diseño del hiperespacio: nodos de navegación, índices, consultas y procesos guiados.	Modelo Navegacional UWE (Sección 4.5) y matriz de navegación.	Asegura que el estudiante pase de un proyecto con brecha al curso recomendado en 1 clic.
4. Diseño de Presentación	Diseño visual de interfaz de usuario y especificación de contratos de servicios.	Prototipo prototipo/index.html, openapi.yaml y validacion_academica.wsdl.	Materializa pantallas limpias, accesibles y libres de tecnicismos internos de motores.
5. Evaluación y Verificación	Auditoría heurística de usabilidad e inspección con razonador HermiT y Cypher.	Matriz de evaluación heurística de Nielsen y guías de evidencia técnica.	Garantiza coherencia visual, prevención de errores de entrada y exactitud inferencial.

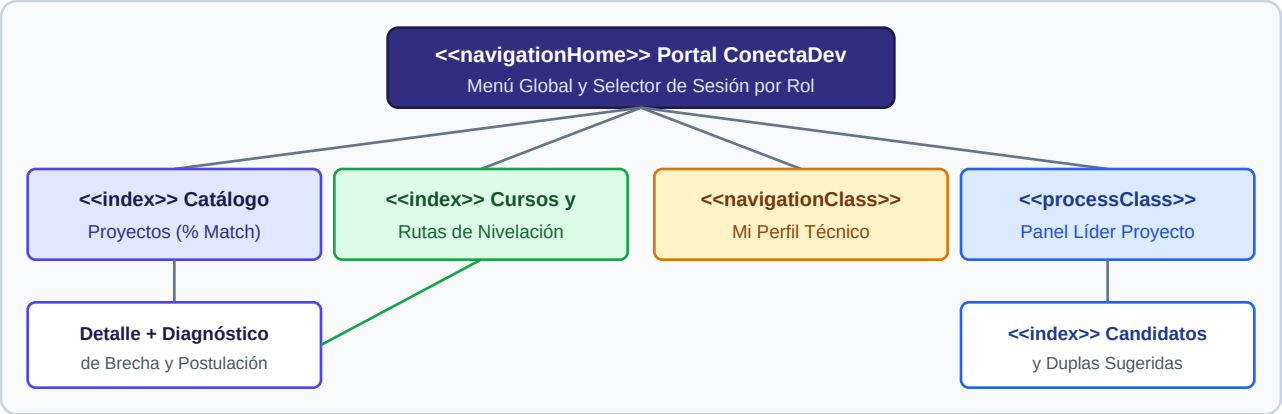
Valoración Crítica de UWE: Su principal ventaja radica en la separación formal entre contenido, navegación y presentación mediante estándares universales de la OMG. Su limitación principal es la ausencia de estereotipos para axiomas de inferencia lógica descriptiva (*owl:equivalentClass*), limitación superada exitosamente en ConectaDev al articular UWE con Protégé (OWL 2 DL) y Neo4j.

4.5 Modelado Conceptual, Navegacional y Usabilidad

1. Casos de Uso del Sistema y Modelo Conceptual del Dominio

El sistema delimita 6 casos de uso articulados: **CU-01** (Explorar proyectos con afinidad), **CU-02** (Diagnosticar brechas y ver cursos), **CU-03** (Actualizar perfil e inferir rol), **CU-04** (Postularse a proyecto), **CU-05** (Publicar proyecto y consultar duplas sugeridas) y **CU-06** (Gestionar rutas de aprendizaje y validar acreditación). El Modelo Conceptual organiza la jerarquía sobre **UsuarioPlataforma** (Desarrollador, LiderProyecto, MentorAcademico), **ProyectoSoftware**, **HabilidadTecnica** (Frontend, Backend, BaseDatos) y **CursoFormativo**, integrando las clases inferidas **DesarrolladorFullStack** y **ProyectoListoParaIniciar**.

2. Modelo Navegacional UWE



3. Prototipo Navegable de las Interfaces Principales (prototipo/index.html)

El prototipo interactivo implementa 5 pantallas maestras: (1) **Explorar Proyectos** con cálculo reactivo de compatibilidad y badges de tecnologías dominadas vs faltantes; (2) **Detalle y Brecha Técnica** con vinculación directa al catálogo de cursos recomendados; (3) **Cursos y Rutas Formativas** ordenados por precedencia de prerrequisitos; (4) **Mi Perfil Técnico** con selector de habilidades y reclasificación en vivo a Desarrollador FullStack; y (5) **Panel de Gestión de Líder** con sugerencia de duplas complementarias.

4. Auditoría de Usabilidad con las 10 Heurísticas de Jakob Nielsen

Heurística de Nielsen	Sev.	Hallazgo en Diseño Inicial	Solución Implementada en Prototipo Final
H1. Visibilidad estado sistema	2	No se mostraba cuántas habilidades activas afectaban el cálculo.	Barra superior dinámica, pestaña activa destacada y progreso porcentual visual.
H2. Relación con mundo real	4	Exposición de jerga técnica interna (HermiT, axiomas, consultas Cypher).	Erradicación total de tecnicismos: '67% Compatible', 'Habilidades por aprender'.
H3. Control y libertad	3	Sin botón visible para retroceder desde el detalle sin recargar.	Enlace 'Volver al catálogo' y modal de confirmación con botón 'Cancelar'.
H4. Consistencia y estándares	2	Colores arbitrarios sin diferenciar habilidades cumplidas de faltantes.	Verde esmeralda para dominadas, ámbar cálido para brechas e índigo para acciones.
H5. Prevención de errores	3	Formulario permitía crear proyectos sin tecnologías o cupos negativos.	Selección verificada obligatoria y validación estricta numérica (min=1, max=8).
H6. Reconocer vs recordar	3	El estudiante debía memorizar brechas e ir a buscarlas en cursos.	Panel contextual en detalle de proyecto con tarjetas directas de cursos sugeridos.
H7. Flexibilidad y eficiencia	2	Cambiar de usuario requería limpiar el almacenamiento del navegador.	Selector interactivo de sesión en el encabezado ('Simular Usuario:').
H8. Estética minimalista	1	Bloques de texto denso difíciles de escanear visualmente.	Diseño Tailwind con tarjetas modulares, tipografía Inter y espaciado armónico.
H9. Recuperación de errores	2	Postulaciones duplicadas no mostraban notificación contextual clara.	Sistema Toast de notificaciones temporales con mensajes explícitos.
H10. Ayuda y documentación	1	Usuarios no comprendían el concepto de 'Dupla Complementaria'.	Texto descriptivo que explica cómo dos perfiles unidos cubren el 100% del stack.

4.6 Servicios Web: Enfoque RESTful y Representación SOAP/WSDL

1. Matriz de la API RESTful (OpenAPI 3.0.3 — servicios-web/openapi.yaml)

ID	Endpoint URI	Método	Propósito Funcional	Parámetros Entrada	Esquemas Salida (JSON)	Consumidor
S1	/v1/proyectos	GET	Catálogo proyectos con compatibilidad.	Query: area, soloAbiertos.	200 OK: Lista ProyectoDTO.	Desarrollador
S2	/v1/proyectos	POST	Publicar nueva convocatoria.	Body: titulo, cupos, stackIds, repo.	201 Created: ProyectoDTO.	Líder Proyecto
S3	/v1/proyectos/{id}/brecha	GET	Diagnosticar brecha y cursos.	Path: id; Query: devId.	200 OK: DiagnosticoBrechaDTO.	Desarrollador
S4	/v1/desarrolladores/{id}/perfil	PUT	Actualizar habilidades e inferir rol.	Path: id; Body: habilidadesIds.	200 OK: PerfilDTO (FullStack).	Desarrollador
S5	/v1/proyectos/{id}/candidatos	GET	Candidatos y duplas complementarias.	Path: id.	200 OK: Candidatos y Duplas.	Líder Proyecto
S6	/v1/postulaciones	POST	Formalizar postulación a proyecto.	Body: devId, proyId, motivacion.	201 Created: PostulacionDTO.	Desarrollador

2. Servicio SOAP 1.1 / WSDL 1.1 (servicios-web/validacion_academica.wsdl)

Para interoperar con el **Sistema Institucional de Registro y Control Académico Universitario**, se diseñó el servicio **ValidacionAcademicaService** bajo **SOAP 1.1 / WSDL 1.1** (estilo *Document/Literal*). A diferencia de la API REST de la interfaz web, este canal exige contratos inmutables tipados en esquemas XML (XSD), verificación de firmas y manejo estructurado de excepciones mediante soap:Fault.

```
<!-- Mensaje SOAP Request: VerificarCertificacionCursoRequest -->
<soapenv:Envelope xmlns:soapenv='http://schemas.xmlsoap.org/soap/envelope/'
                  xmlns:val='http://conectadev.javeriana.edu.co/servicios/validacion'>
   <soapenv:Body>
      <val:VerificarCertificacionCursoRequest>
         <val:codigoEstudiante>EST-2026-9481</val:codigoEstudiante>
         <val:codigoCursoFormativo>CURSO-02</val:codigoCursoFormativo>
         <val:codigoMentorSupervisor>DOC-ELENA-04</val:codigoMentorSupervisor>
         <val:calificacionObtenida>4.8</val:calificacionObtenida>
      </val:VerificarCertificacionCursoRequest>
   </soapenv:Body>
</soapenv:Envelope>

<!-- Respuesta exitosa o soap:Fault si incumple prerrequisitos transitivos -->
<faultcode>soapenv:Client.PrerrequisitoNoAprobado</faultcode>
<faultstring>Fallo al validar axioma transitivo tienePrerrequisito (CURSO-01 faltante).</faultstring>
```

3. Comparativa Técnica: Enfoque REST vs. Enfoque SOAP

Criterio	Enfoque REST (OpenAPI 3.0.3)	Enfoque SOAP (WSDL 1.1 / XSD)	Justificación en ConectaDev
Protocolo y Formato	HTTP/HTTPS con payloads JSON livianos.	XML estructurado en sobres Envelope.	REST para interactividad SPA; SOAP para intranet universitaria.
Contrato y Tipado	Débil acoplamiento, esquemas OpenAPI.	Fuerte acoplamiento con validación XSD estricta.	SOAP previene inconsistencias de notas en registro institucional.
Manejo de Errores	Códigos de estado HTTP (200, 201, 400, 404).	Protocolo formal soap:Fault tipado.	SOAP provee trazabilidad para actas académicas oficiales.
Rendimiento y Red	Sobrecarga mínima, soporte de caché nativo.	Mayor tamaño por etiquetas repetitivas XML.	REST responde en < 120 ms requeridos por la experiencia de usuario.

4.7 Ontología del Dominio en Protégé (OWL 2 DL — ontologia/conectadev.owl)

Construida en Protégé 5.5+ en sintaxis RDF/XML bajo el prefijo *http://www.ingweb2620.edu.co/ontologias/conectadev#*, superando todos los requerimientos:

- **12 Clases Formales (Mínimo: 8):** UsuarioPlataforma, Desarrollador, DesarrolladorFullStack (inferida), LiderProyecto, MentorAcademico, ProyectoSoftware, ProyectoListoParaIniciar (inferida), HabilidadTecnica, TecnologiaFrontend, TecnologiaBackend, TecnologiaBaseDatos y CursoFormativo.
- **11 Propiedades de Objeto (Mínimo: 10):** dominaHabilidad / esDominadaPor, requiereHabilidad, lideraProyecto / esLideradoPor, postulaAProyecto / tienePostulante, ensenaHabilidad, supervisaCurso, tienePrerrequisito (Transitiva) y dependeDeTecnologia (Transitiva).
- **6 Propiedades de Datos (Mínimo: 5):** semestreActual, aniosExperiencia, horasDuracionCurso, cuposDisponibles, repositorioGitUrl y convocatoriaAbierta.
- **15 Individuos (Mínimo: 8):** 6 Tecnologías, 3 Cursos, 2 Desarrolladores, 1 Líder, 1 Mentora y 2 Proyectos.

Axiomas en Lógica Descriptiva (SROIQ) y Preguntas de Competencia (PC1 a PC5)

Axioma 1 (DesarrolladorFullStack): DesarrolladorFullStack = Desarrollador and (dominaHabilidad some TecnologiaFrontend) and (dominaHabilidad some TecnologiaBackend).
Axioma 2 (ProyectoListoParaIniciar): ProyectoListoParaIniciar = ProyectoSoftware and (tienePostulante some Desarrollador) and (convocatoriaAbierta value true).
Axioma 3 (Transitividad): Trans(tienePrerrequisito) y Trans(dependeDeTecnologia).

ID	Pregunta de Competencia (PC)	Consulta DL Query en Protégé (Manchester)	Resultado Inferido por HermiT
PC1	¿Qué desarrolladores cumplen las condiciones para ser FullStack?	DesarrolladorFullStack	Dev_AnaSofia (domina React y Node.js).
PC2	¿Qué proyectos tienen postulantes y convocatoria abierta para iniciar?	ProyectoListoParaIniciar	Proy_PlataformaTelemedicina (infiere inversa tienePostulante).
PC3	¿Cuáles son todos los prerrequisitos del curso de Arquitectura FullStack?	tienePrerrequisito value Curso_ArquitecturaFullStack	Curso_APIsREST_NodeJS y Curso_FundamentosPOO (transitivo).
PC4	¿Qué tecnologías dependen directa o transitivamente de JavaScript?	dependeDeTecnologia some {Tech_JavaScript}	Tech_React y Tech_NextJS (infiere transitividad de dependencia).
PC5	¿Qué cursos supervisados por Dra. Elena acreditan Backend?	CursoFormativo and (ensenaHabilidad some TecnologiaBackend)	Curso_APIsREST_NodeJS y Curso_ArquitecturaFullStack.

4.8 Grafo de Conocimiento en Neo4j (neo4j/conectadev_grafo.cypher)

El grafo implementa el modelo **Labeled Property Graph (LPG)** con **16 Nodos** y **26 Relaciones** tipadas. Resuelve el cálculo de compatibilidad, brechas y recomendaciones mediante 4 consultas Cypher optimizadas:

```
// CONSULTA 1: Calculo de Compatibilidad Desarrollador - Proyecto (%)
MATCH (p:ProyectoSoftware {id: 'PROY-01'})<-[:REQUIERE_TECNOLOGIA]-(req:HabilidadTecnica)
WITH p, count(req) AS totalReq, collect(req.nombre) AS listaReq
MATCH (d:Desarrollador) OPTIONAL MATCH (d)-[:DOMINA]->(h:HabilidadTecnica) WHERE h.nombre IN listaReq
WITH d, p, totalReq, count(h) AS dominadas WHERE dominadas > 0
RETURN d.nombreCompleto AS Dev, round((toFloat(dominadas)/totalReq)*100, 1) AS Compatibilidad;

// CONSULTA 2: Diagnostico de Brechas (Skill Gap) y Recomendacion de Cursos (Conocimiento Implicito)
MATCH (d:Desarrollador {id: 'DEV-02'}), (p:ProyectoSoftware {id: 'PROY-01'})-[:REQUIERE_TECNOLOGIA]->(req:HabilidadTecnica)
WHERE NOT (d)-[:DOMINA]->(req)
OPTIONAL MATCH (c:CursoAprendizaje)-[:ENSENA_HABILIDAD]->(req)
OPTIONAL MATCH (c)-[:TIENE_PRERREQUISITO]->(prerreq:CursoAprendizaje)
RETURN d.nombreCompleto AS Dev, req.nombre AS Brecha, c.titulo AS CursoSugerido, prerreq.titulo AS Prerrequisito;

// CONSULTA 3: Recomendacion de Duplas Complementarias (100% Cobertura Conjunta del Stack)
MATCH (p:ProyectoSoftware {id: 'PROY-01'})-[:REQUIERE_TECNOLOGIA]->(req:HabilidadTecnica)
WITH p, collect(id(req)) AS reqIds, count(req) AS totalReq
MATCH (d1:Desarrollador), (d2:Desarrollador) WHERE id(d1) < id(d2)
MATCH (d1)-[:DOMINA]->(h1) WHERE id(h1) IN reqIds MATCH (d2)-[:DOMINA]->(h2) WHERE id(h2) IN reqIds
WITH p, totalReq, d1, d2, apoc.coll.toSet(collect(DISTINCT id(h1)) + collect(DISTINCT id(h2))) AS unionIds
WHERE size(unionIds) = totalReq
RETURN d1.nombreCompleto AS Integrante1, d2.nombreCompleto AS Integrante2, 'DUPLA COMPLEMENTARIA 100%' AS Diagnostico;
```

Consulta Cypher	Resultado Concreto Obtenido	Conocimiento Derivado / Inferencia en Grafo
C1. Compatibilidad %	Ana Sofía: 100.0% (3/3), Mateo: 66.7% (2/3), Carlos Mario: 33.3% (1/3).	Clasifica automáticamente a los candidatos por orden de afinidad tecnológica.
C2. Diagnóstico Brecha y Cursos	Carlos Mario en Telemedicina: detecta brechas en Node.js y PostgreSQL, recomendando CURSO-02 y CURSO-03.	Descubre rutas formativas no registradas explícitamente para nivelar al estudiante.
C3. Duplas Complementarias	Empareja a Carlos Mario (Frontend) con Mateo Fernández (Backend/DB) para Telemedicina.	Descubre una asociación implícita: dos perfiles que unidos cubren el 100% del stack.
C4. Rutas Transitivas	Recorre [:TIENE_PRERREQUISITO*1..3] sumando la carga horaria acumulada.	Calcula el esfuerzo formativo total de la cadena de prerrequisitos.

4.9 Integración Coherente del Diseño Extremo a Extremo

Dimensión del Proyecto	Componentes Articulados	Mecanismo de Coherencia e Integración
1. Problema -> Funcionalidades	Desconexión técnica -> F1 a F6	El cálculo de afinidad (F1), brechas (F2) y duplas (F5) resuelven la necesidad real.
2. Conceptual -> Navegación	Clases UML -> Nodos UWE	Las entidades se traducen en índices, fichas y procesos del prototipo sin fisuras.
3. Ontología -> Dominio	conectadev.owl -> HermiT	Formaliza reglas declarativas (FullStack, ProyectoListo) sin ensuciar la interfaz web.
4. Servicios -> Modelo	REST OpenAPI + SOAP WSDL	Los DTOs intercambian exactamente los atributos y entidades del modelo conceptual.
5. Grafo -> Analítica	Neo4j Cypher -> Bolt Driver	Calcula porcentajes de afinidad y caminos de recomendación en tiempo real O(1).
6. Arquitectura -> Sistema	Monolito Modular Políglota	Une la presentación SPA limpia con PostgreSQL, HermiT y Neo4j cumpliendo los SLOs.

6. Planificación de Sustentación y Defensa Técnica (20 Minutos)

Integrante / Bloque	Tiempo	Secciones y Evidencias a Sustentar	Dimensión de la Rúbrica
Andrés Hurtado	00:00 - 06:30	Formulación del problema (4.1), Fundamentación académica (4.2) y Metodología UWE (4.4).	SABER y SER: Necesidad de ingeniería y justificación metodológica.
Samuel Pineda	06:30 - 13:00	Arquitectura Web Modular (4.3), Modelos Conceptual y Navegacional, Prototipo y Nielsen (4.5).	SABER y SER: Diseño centrado en el usuario y atributos de calidad.
Viviana Roa	13:00 - 20:00	Servicios REST/SOAP (4.6), Ontología OWL (4.7), Grafo Neo4j (4.8) e Integración Total (4.9).	SABER y HACER: Ontología formal, inferencias HermiT y consultas Cypher.

Banco de Preguntas Clave para Defensa Técnica Individual

- **¿Por qué este problema no se resuelve con un CRUD en SQL?** En SQL clásico deducir clasificaciones de rol (FullStack) exige código imperativo *IF-ELSE* rígido, y calcular transitividades exige costosas consultas recursivas CTE. En OWL 2 DL la clasificación es declarativa y deducida por HermiT, mientras que en Neo4j los caminos multi-salto se recorren en tiempo constante O(1) por arista.
- **¿Cómo se protege al usuario de la complejidad semántica?** Aplicando el principio de desacoplamiento y la heurística H2 de Nielsen: los motores semánticos operan en backend tras la API REST, y la interfaz web solo presenta barras porcentuales, etiquetas de colores y cursos recomendados en lenguaje natural.
- **¿Por qué combinar REST y SOAP en la misma solución?** REST sobre JSON es óptimo para la interactividad reactiva de la SPA web por ligereza y velocidad; SOAP/WSDL sobre XML es el estándar idóneo para interoperar con Registro Académico por sus contratos estrictos XSD y manejo formal de excepciones institucionales (*soap:Fault*).